

Music genre classification: comparing instance-based learning (KNN) with a deep neural network for audio-feature-based genre classification

A Machine Learning Mini-Project Report

Submitted in partial fulfilment of coursework requirements

Archana Angel 2414110705 / 34

Kuldeep Singh Rathore 2414110682 / 17

Department of Electronics and Communication Engineering

Bharati Vidyapeeth (Deemed to be University) College of Engineering, Pune

Abstract

This project addresses the task of automatically classifying music tracks into genres using features extracted directly from audio signals. Using the GTZAN dataset (999 usable 30-second clips across 10 genres), a set of 55 timbral, spectral, and rhythmic features was extracted per clip using the librosa audio-processing library. Two fundamentally different learning paradigms were then trained and compared on these features: an instance-based ("lazy learning") algorithm, K-Nearest Neighbours (KNN), and an eager, parametric deep neural network (DNN). The DNN achieved a test accuracy of 70.0% against 66.5% for KNN, with both models showing complementary strengths across different genres. The results are analysed in terms of accuracy, per-genre precision/recall, confusion patterns, and computational cost, to draw out the practical trade-offs between the two paradigms.

Introduction

Problem statement

Music genre classification is a classic Music Information Retrieval (MIR) problem: given an audio recording, predict which genre it belongs to (e.g. blues, classical, jazz, rock). It is a non-trivial task because genre boundaries are inherently fuzzy and subjective, and because raw audio waveforms are extremely high-dimensional and not directly usable by most classical machine learning algorithms. The standard approach, followed in this project, is to first convert each audio clip into a compact set of numerical features that summarise its timbral, spectral, and rhythmic characteristics, and to then apply a classifier on top of these features.

Objective

- Extract a meaningful set of audio features from raw music clips using signal-processing techniques.
- Train an instance-based learning algorithm (K-Nearest Neighbours) on these features.

- Train a deep neural network on the same features for a fair, controlled comparison.
- Evaluate and compare both models on accuracy, per-class performance, and computational cost.
- Draw conclusions about when each learning paradigm is preferable for this kind of task.

Dataset

The GTZAN dataset was used, which is the most widely-cited benchmark dataset for music genre classification. It consists of 1000 audio tracks, each 30 seconds long, evenly split across 10 genres: blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, and rock (100 clips per genre). One file (jazz.00054.wav) is a known corrupted file within the public distribution of GTZAN and was automatically skipped during feature extraction, leaving 999 usable clips.

Methodology

Feature extraction

Each audio clip was loaded as a mono waveform at 22.05 kHz using librosa, and the following features were computed:

- **Mel-Frequency Cepstral Coefficients (MFCCs, 20 coefficients)** — capture the timbral texture of the sound, and are the single most discriminative feature family for this task.
- **Chroma STFT** — captures the distribution of energy across the 12 pitch classes, reflecting harmonic/tonal content.
- **Spectral centroid, bandwidth, and rolloff** — describe the “brightness” and spread of the frequency content.
- **Zero-crossing rate** — a measure of signal noisiness, useful for distinguishing percussive/noisy genres from tonal ones.
- **RMS energy** — a measure of loudness.
- **Tempo (beats per minute)** — genres often cluster around characteristic tempo ranges.

Because most of these features vary continuously over the duration of a clip, the mean and variance of each time-varying feature were taken across the full 30-second clip, collapsing it into a single fixed-length feature vector of 55 values per track. This mirrors the standard approach used in the original GTZAN feature-based benchmarks.

Instance-based learning: K-nearest neighbours

KNN is a non-parametric, instance-based ("lazy") learning algorithm: it does not learn an internal model during training — it simply stores the entire training set. At prediction time, it measures the distance (in feature space) between a new sample and every stored training sample, and assigns the majority genre among the k closest ones. Feature scaling (standardisation) was applied before training, since KNN's distance calculation is sensitive to the relative scale of different features. A sweep over $k = 1$ to 25 was performed to find the value that maximised test accuracy; distance-weighted voting was used so that closer neighbours have proportionally more influence than farther ones.

Deep neural network

In contrast, the DNN is an eager, parametric learner: it processes the training data through repeated passes (epochs) and gradually adjusts a fixed set of internal weights via backpropagation, so that once training is complete, the original training data is no longer needed to make predictions. The network used here is a fully-connected feed-forward architecture with three hidden layers (256, 128, and 64 units, each with ReLU activation, batch normalisation, and dropout for regularisation), followed by a 10-way softmax output layer. It was trained using the Adam optimiser with early stopping (monitoring validation loss, patience of 12 epochs) to prevent overfitting.

Train/test split and evaluation

The dataset was split into 80% training (799 clips) and 20% held-out testing (200 clips), using stratified sampling to preserve genre balance in both sets. Both models were evaluated on the identical test set, using accuracy, per-genre precision/recall/F1-score, confusion matrices, and training/inference time as comparison metrics.

Results

KNN performance

The best result was obtained at $k = 15$ neighbours (with distance weighting), achieving a test accuracy of 66.5%. Fig. 1 shows how accuracy varied across the sweep of k values tested.

Table 1 shows the full per-genre classification report for the best KNN model.

Fig. 2 presents the confusion matrix for the KNN classifier.

Deep neural network performance

The DNN achieved a test accuracy of 70.0%, with early stopping halting training once validation loss stopped improving. Fig. 3 shows the training and validation loss/accuracy curves.

Table 2 shows the full per-genre classification report for the DNN.

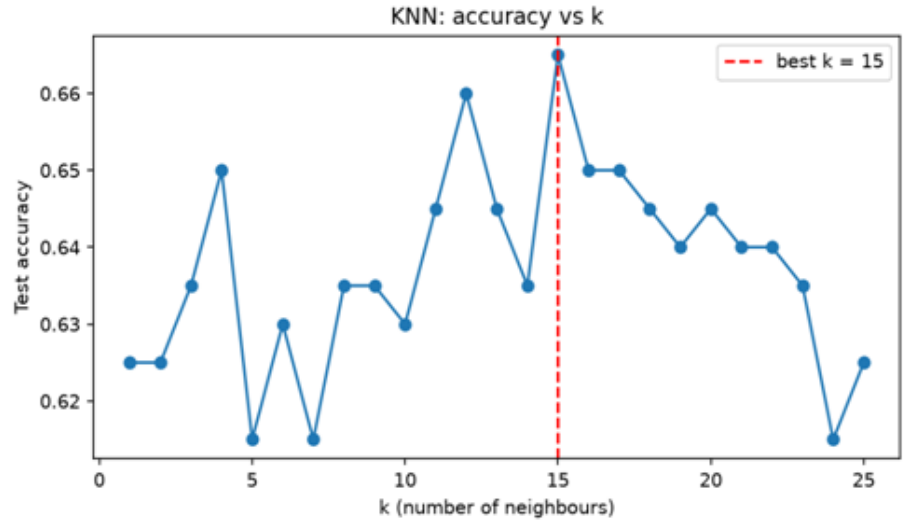


Fig 1. KNN test accuracy as a function of k (number of neighbours).

Table 1. KNN ($k = 15$) precision, recall, and F1-score per genre.

Genre	Precision	Recall	F1-score	Support
blues	0.78	0.35	0.48	20
classical	0.90	0.90	0.90	20
country	0.57	0.80	0.67	20
disco	0.57	0.60	0.59	20
hiphop	0.71	0.50	0.59	20
jazz	0.74	0.70	0.72	20
metal	0.89	0.80	0.84	20
pop	0.59	0.85	0.69	20
reggae	0.50	0.50	0.50	20
rock	0.59	0.65	0.62	20

Table 2. Deep Neural Network precision, recall, and F1-score per genre.

Genre	Precision	Recall	F1-score	Support
blues	0.67	0.60	0.63	20
classical	0.95	0.95	0.95	20
country	0.68	0.75	0.71	20
disco	0.50	0.65	0.57	20
hiphop	0.55	0.60	0.57	20
jazz	0.67	0.90	0.77	20
metal	1.00	0.80	0.89	20
pop	0.88	0.70	0.78	20
reggae	0.60	0.60	0.60	20
rock	0.69	0.45	0.55	20

Fig. 4 presents the confusion matrix for the Deep Neural Network.

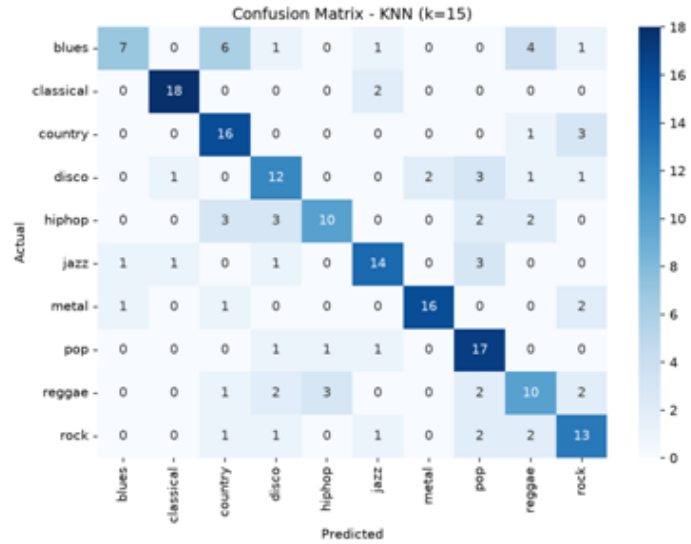


Figure 2: Confusion matrix for the KNN classifier.

Fig 2. Confusion matrix for the KNN classifier.

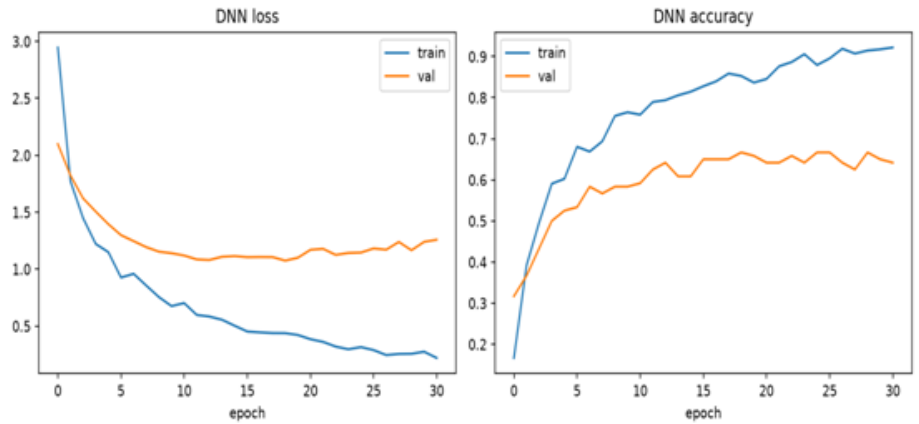


Fig 3. DNN training/validation loss and accuracy across epochs. The widening gap between training and validation accuracy after around epoch 10 indicates mild overfitting, which early stopping and dropout helped to contain.

Comparative analysis

Table 3 and Fig. 5 summarise the head-to-head comparison between the two models.

Discussion

Overall accuracy

The DNN outperformed KNN by 3.5 percentage points overall (70.0% vs. 66.5%), and by a similar margin on macro-averaged F1-score (0.70 vs. 0.66).

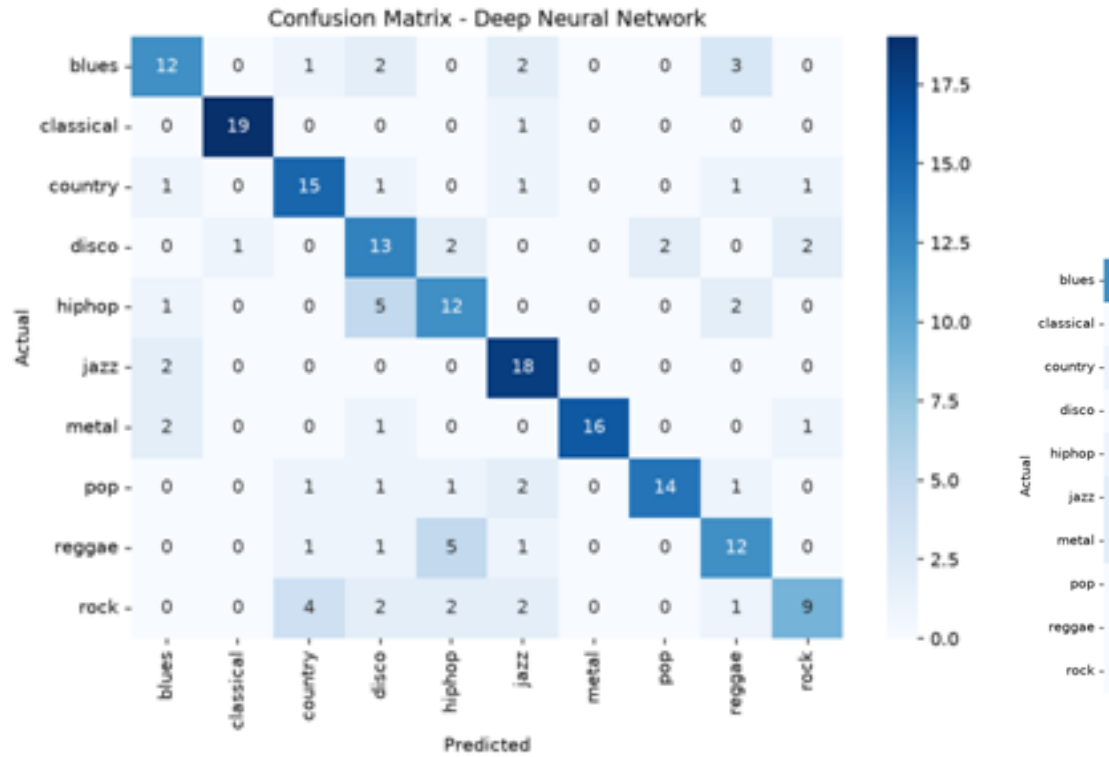


Figure 4: Confusion matrix for the Deep Neural Network.

Fig 4. Confusion matrix for the Deep Neural Network.

Table 3. Summary comparison of KNN and the Deep Neural Network.

Metric	KNN ($k = 15$)	Deep Neural Network	Better
Test Accuracy	66.5%	70.0%	DNN
Macro Avg. F1-score	0.66	0.70	DNN
Weighted Avg. F1-score	0.66	0.70	DNN
Training Time	0.001 s	7.570 s	KNN
Inference Time (200 samples)	0.0101 s	0.2231 s	KNN

This is consistent with the general expectation that a parametric model, which can learn non-linear combinations of the 55 input features, can capture more complex decision boundaries than an instance-based method relying purely on local distance in the raw feature space. The margin is modest rather than dramatic, however, which is itself a meaningful result: GTZAN is a well-known, noisy benchmark with fuzzy genre boundaries and some mislabeled tracks, so no method achieves near-perfect accuracy on it, and the relatively small performance gap suggests that the quality of the extracted features matters at least as much as the sophistication of the classifier.

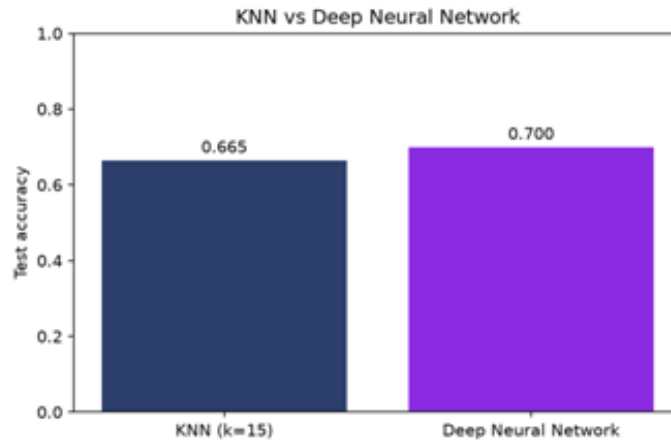


Figure 5: Test accuracy comparison between KNN and the Deep Neural Network.

Fig 5. Test accuracy comparison between the KNN and the Deep Neural Network.

Per-genre patterns

Both models performed strongly on classical and metal (F1-scores of 0.90 and 0.84 for KNN; 0.95 and 0.89 for the DNN), which have highly distinctive timbral signatures that are easy to separate even with simple distance-based matching. Both models struggled most with reggae (F1-scores of 0.50 and 0.60 respectively), which acoustically overlaps with several other genres in this feature space. A notable exception to the DNN's general advantage is rock, where KNN achieved a higher F1-score (0.62 vs. 0.55). The DNN's confusion matrix shows that it frequently misclassified rock as country, indicating that the DNN's learned decision boundary, while better overall, made a specific trade-off that reduced its ability to separate these two genres in exchange for gains elsewhere (for example, jazz recall improved from 0.70 with KNN to 0.90 with the DNN).

Computational trade-offs

The training/inference timing results illustrate the fundamental conceptual difference between the two paradigms. KNN's "training" time was effectively zero (0.001 seconds) because it performs no learning at training time — it simply stores the data — while its inference time (0.0101 seconds for 200 test samples) reflects the cost of comparing each test sample against the entire training set. The DNN shows the opposite pattern: it required 7.57 seconds of upfront training to adjust its weights via backpropagation, after which inference was comparatively simple (0.2231 seconds for 200 samples, using a single forward pass per sample). In a deployment scenario with a very large training set, this trade-off would become more significant: KNN's inference cost grows with the size of the training set, whereas a trained DNN's inference cost stays constant regardless of how much data it was trained on.

Conclusion

This project demonstrated a complete pipeline for audio-based music genre classification: feature extraction from raw audio using librosa, followed by training and evaluating two contrasting learning paradigms. The deep neural network achieved higher overall accuracy (70.0% vs. 66.5%) and better macro-averaged F1-score, confirming that a parametric model can learn more nuanced decision boundaries from the same feature set. However, KNN remained competitive and even outperformed the DNN on specific genres (notably rock), while requiring negligible training time and a much simpler implementation. This suggests that the choice between instance-based and deep-learning approaches for this kind of task should depend on the specific constraints of the deployment scenario — e.g., available training data, need for fast retraining as new data arrives, and interpretability requirements — rather than defaulting to the more complex model on the assumption that it is always superior.

References

1. Tzanetakis, G., & Cook, P. (2002). Musical genre classification of audio signals. *IEEE Transactions on Speech and Audio Processing*, 10(5), 293–302.
2. McFee, B., et al. (2015). librosa: Audio and music signal analysis in Python. *Proceedings of the 14th Python in Science Conference*.
3. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
4. Chollet, F., et al. (2015). Keras. <https://keras.io>
5. GTZAN Genre Collection dataset, as distributed via Kaggle (andradaolteanu/gtzan-dataset-music-genre-classification).